



System Architecture - Complete Technical Explanation



High-Level Overview



				Approvals/	(Approval logs)		
				Temp/	(Pending updates)		
				Updated Doc/	(Approved updates)		
				Catalog System Map/	(Contact info)		
				Vault/Chats/	(Chat history)		
				YYYY-MM-DD/	(Date-organized)		

Complete Request Flow

Step-by-Step: What Happens When You Ask a Question

1. USER TYPES MESSAGE
↓
2. FRONTEND CAPTURES INPUT
↓
3. FRONTEND SENDS HTTP POST TO BACKEND
↓
4. BACKEND RECEIVES REQUEST
↓
5. BACKEND SEARCHES VAULT FILES
↓
6. BACKEND BUILDS CONTEXT
↓
7. BACKEND SENDS TO AI (GROQ/OLLAMA)
↓
8. AI PROCESSES & GENERATES RESPONSE
↓
9. BACKEND STREAMS RESPONSE TO FRONTEND
↓
10. FRONTEND DISPLAYS RESPONSE
↓
11. BACKEND SAVES CHAT HISTORY

Frontend (index.html) - Detailed Breakdown

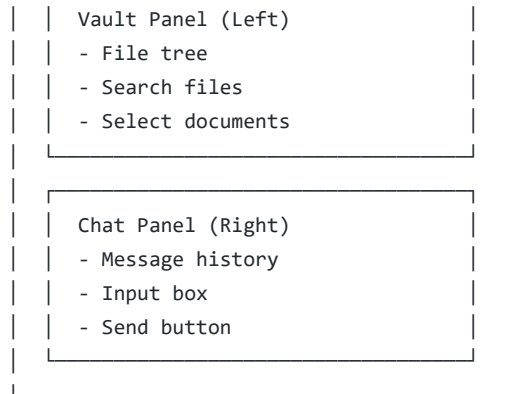
Location:

```
c:\Users\adisba\OneDrive - amazon.com\PROJECT UI\Chat UI\obsidian-chat\frontend\index.html
```

What It Does:

1. User Interface

				CHAT UI		



2. JavaScript Functions

Key Functions:

```
// Send message to backend
async function sendMessage() {
  const message = document.getElementById('user-input').value;

  // POST to backend
  const response = await fetch('http://localhost:3003/api/chat', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      message: message,
      selectedFile: currentFile,
      history: conversationHistory,
      chatId: currentChatId
    })
  });
};

// Stream response
const reader = response.body.getReader();
// ... handle streaming
}

// Load vault files
async function loadVault() {
  const response = await fetch('http://localhost:3003/api/vault');
  const data = await response.json();
  // Display file tree
}

// Load specific file
async function loadFile(filePath) {
  const response = await fetch(`http://localhost:3003/api/file?path=${filePath}`);
  const data = await response.json();
  // Display file content
}
```

3. Server-Sent Events (SSE)

Live Updates:

```
// Connect to SSE endpoint
const eventSource = new EventSource('http://localhost:3003/api/events');

// Listen for vault changes
eventSource.addEventListener('message', (event) => {
  const data = JSON.parse(event.data);

  if (data.type === 'vault-changed') {
    // Reload vault files
    loadVault();
  }

  if (data.type === 'approval-added') {
    // Show notification
    showNotification('New approval request');
  }
});
```

4. Markdown Rendering

Libraries Used:

- **marked.js** - Converts Markdown to HTML
- **highlight.js** - Syntax highlighting for code blocks

```
// Render markdown
function renderMarkdown(text) {
  return marked.parse(text, {
    highlight: function(code, lang) {
      return hljs.highlightAuto(code).value;
    }
  });
}
```

Backend (server.js) - Detailed Breakdown

Location:

```
c:\Users\adisba\OneDrive - amazon.com\PROJECT UI\Chat UI\obsidian-chat\backend\server.js
```

Technology Stack:

- **Node.js** - JavaScript runtime
- **Express.js** - Web framework
- **axios** - HTTP client for API calls
- **fs-extra** - File system operations

Server Initialization:

```
const express = require('express');
const app = express();
const PORT = 3003;
const HOST = '0.0.0.0'; // Listen on all network interfaces

// Middleware
app.use(cors()); // Allow cross-origin requests
app.use(express.json({ limit: '50mb' })); // Parse JSON bodies
app.use(express.static(path.join(__dirname, '../frontend'))); // Serve frontend

// Start server
app.listen(PORT, HOST, () => {
  console.log(`🚀 Server running on http://127.0.0.1:${PORT}`);
});
```

API Endpoints:

1. GET /api/vault - List All Files

```
app.get('/api/vault', (req, res) => {
  // Walk through Obsidian database directory
  const files = [];

  function walkDir(dir) {
    const items = fs.readdirSync(dir);
    items.forEach(item => {
      const fullPath = path.join(dir, item);
      const stat = fs.statSync(fullPath);

      if (stat.isDirectory()) {
        walkDir(fullPath); // Recursive
      } else if (item.endsWith('.md')) {
        files.push({
          name: item,
          path: relativePath,
          type: 'file'
        });
      }
    });
  }

  walkDir(OBSIDIAN_DB_PATH);
  res.json({ files });
});
```

2. GET /api/file?path=... - Get File Content

```

app.get('/api/file', (req, res) => {
  const filePath = req.query.path;
  const fullPath = path.join(OBSIDIAN_DB_PATH, filePath);

  // Security check
  if (!fullPath.startsWith(OBSIDIAN_DB_PATH)) {
    return res.status(403).json({ error: 'Access denied' });
  }

  // Read file
  const content = fs.readFileSync(fullPath, 'utf-8');

  res.json({
    name: path.basename(filePath),
    content: content
  });
});

```

3. POST /api/chat - Main Chat Endpoint

This is the most complex endpoint:

```

app.post('/api/chat', async (req, res) => {
  const { message, context, history, selectedFile, chatId } = req.body;

  // — STEP 1: Validate Input —
  if (!message || message.trim() === '') {
    return res.status(400).json({ error: 'Message required' });
  }

  // — STEP 2: Search Vault Files —
  const vaultResults = searchVaultForQuestion(message, selectedFile);

  // — STEP 3: Build Context —
  const smartContext = context ? context.substring(0, 8000) : null;
  const vaultContext = vaultResults
    ? vaultResults.map(r => `${r.file}]\n${r.snippet}`).join('\n\n')
    : null;

  // — STEP 4: Determine Query Type —
  const isGreeting = /^(hi|hello|hey)$/i.test(message.trim());
  const isNormalizationQuery = /(blk|red|gps|normalization)/i.test(message);

  // — STEP 5: Build System Prompt —
  let systemPrompt = '';
  if (isNormalizationQuery) {
    systemPrompt = `You are an expert in catalog normalization...
    ${smartContext ? `\n--- CONTENT ---\n${smartContext}` : ''}
    ${vaultContext ? `\n--- VAULT ---\n${vaultContext}` : ''}`;
  }
}

```

```

// — STEP 6: Build Conversation History —
let conversationHistory = history || [];
conversationHistory.push({ role: 'user', content: message });

// Keep only last 4 messages
if (conversationHistory.length > 4) {
  conversationHistory = conversationHistory.slice(-4);
}

// — STEP 7: Try Groq API First —
const groqMessages = [
  { role: 'system', content: systemPrompt },
  ...conversationHistory
];

try {
  const groqRes = await axios.post(GROQ_API_URL, {
    model: 'llama-3.1-8b-instant',
    messages: groqMessages,
    stream: true,
    max_tokens: 1024,
    temperature: 0.3
  }, {
    headers: { 'Authorization': `Bearer ${GROQ_API_KEY}` },
    responseType: 'stream'
  });

  // — STEP 8: Stream Response to Frontend —
  res.setHeader('Content-Type', 'text/event-stream');

  let aiResponse = '';
  groqRes.data.on('data', (chunk) => {
    // Parse streaming response
    const token = extractToken(chunk);
    aiResponse += token;

    // Send to frontend
    res.write(`data: ${JSON.stringify({ token })}\n\n`);
  });

  groqRes.data.on('end', () => {
    // — STEP 9: Save Chat History —
    saveChatToFile(message, aiResponse, chatId);

    // — STEP 10: Send Done Signal —
    res.write(`data: ${JSON.stringify({ done: true })}\n\n`);
    res.end();
  });
} catch (groqErr) {
  // — STEP 11: Fallback to Ollama —
  if (groqErr.response?.status === 429) {

```

```

        console.log('⚠️ Groq rate limited – falling back to Ollama');
        await useOllama();
    }
}
});

```

4. GET /api/events - Server-Sent Events

Live Updates:

```

const sseClients = new Set();

app.get('/api/events', (req, res) => {
  // Set SSE headers
  res.setHeader('Content-Type', 'text/event-stream');
  res.setHeader('Cache-Control', 'no-cache');
  res.setHeader('Connection', 'keep-alive');

  // Add client to set
  sseClients.add(res);

  // Send heartbeat every 20s
  const ping = setInterval(() => {
    res.write('data: {"type":"ping"}\n\n');
  }, 20000);

  // Remove client on disconnect
  req.on('close', () => {
    clearInterval(ping);
    sseClients.delete(res);
  });
});

// Broadcast to all clients
function broadcast(payload) {
  const data = `data: ${JSON.stringify(payload)}\n\n`;
  sseClients.forEach(client => {
    client.write(data);
  });
}

// Watch vault folder for changes
fs.watch(OBSIDIAN_DB_PATH, (eventType, filename) => {
  if (filename && filename.endsWith('.md')) {
    broadcast({ type: 'vault-changed', file: filename });
  }
});

```

Groq API (Primary)

Connection:

```
const GROQ_API_KEY = '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';
const GROQ_API_URL = 'https://api.groq.com/openai/v1/chat/completions';
const GROQ_MODEL = 'llama-3.1-8b-instant';

// Make request
const response = await axios.post(GROQ_API_URL, {
  model: GROQ_MODEL,
  messages: [
    { role: 'system', content: systemPrompt },
    { role: 'user', content: userMessage }
  ],
  stream: true,
  max_tokens: 1024,
  temperature: 0.3
}, {
  headers: {
    'Authorization': `Bearer ${GROQ_API_KEY}`,
    'Content-Type': 'application/json'
  },
  responseType: 'stream'
});
```

Why Groq?

-  Fast (2-5 seconds)
-  Cloud-based (no local resources)
-  Free tier (6,000 requests/day)
-  Rate limited
-  Requires internet

Ollama (Fallback)

Connection:

```
const OLLAMA_BASE_URL = 'http://127.0.0.1:11434';
const OLLAMA_API = `${OLLAMA_BASE_URL}/api/generate`;

// Make request
const response = await axios.post(OLLAMA_API, {
  model: 'mistral',
  prompt: fullPrompt,
  stream: true,
  options: {
    temperature: 0.7,
    num_predict: 400,
    num_ctx: 2048
  }
}, {
```

```
    timeout: 60000,  
    responseType: 'stream'  
  });
```

Why Ollama?

-  Unlimited requests
-  Complete privacy (local)
-  No internet needed
-  Slower (30-60 seconds)
-  Requires local installation
-  Uses 4-8GB RAM

Fallback Logic:

```
try {  
  // Try Groq first  
  await useGroq();  
} catch (groqErr) {  
  if (groqErr.response?.status === 429) {  
    // Rate limited - use Ollama  
    console.log('⚠️ Groq rate limited – falling back to Ollama');  
    await useOllama();  
  } else {  
    // Other error - use Ollama  
    console.error('⚠️ Groq failed:', groqErr.message);  
    await useOllama();  
  }  
}
```

File System Structure

Directory Layout:

```
PROJECT UI/Chat UI/  
|  
├─ obsidian-chat/  
|   ├─ backend/  
|   |   ├─ server.js      ← Backend server  
|   |   ├─ package.json   ← Dependencies  
|   |   └─ node_modules/  ← Installed packages  
|   |  
|   └─ frontend/  
|       └─ index.html      ← Frontend UI  
|  
└─ Vault/  
    ├─ Chats/              ← Saved conversations  
    |   └─ YYYY-MM-DD/    ← Date folders  
    |       └─ chat_*.md  ← Chat files
```

```
└─ Obsidian database/
  │ └─ SOP/                                ← Source documents
  │   │ └─ Starfish Modular Title Quality Audits.md
  │   │ └─ Starfish Vanguard SOP V7.md
  │   └─ Publishing SOP.md
  │
  │ └─ Catalog System Map _ System Details.md
  │
  │ └─ Approvals/                          ← Approval logs
  │   └─ YYYY-MM-DD.md                    ← Daily logs
  │
  │ └─ Temp/                               ← Pending updates
  │   └─ *_temp_*.md                      ← Temp documents
  │
  │ └─ Updated Doc/                        ← Approved updates
  │   └─ *_Approved_*.md                  ← Final versions
  │
  └─ file-converter/                      ← Separate service
    └─ backend/
      └─ server.js                        ← Port 3002
```

Data Flow Examples

Example 1: Simple Question

```
USER: "Is blk correct?"
↓
FRONTEND: POST /api/chat
{
  message: "Is blk correct?",
  selectedFile: null,
  history: []
}
↓
BACKEND:
1. Detect query type: normalization
2. Search vault: Find "Starfish Modular Title Quality Audits.md"
3. Extract relevant sections
4. Build system prompt with context
5. Send to Groq API
↓
GROQ API:
1. Process with llama-3.1-8b-instant
2. Generate response
3. Stream back tokens
↓
BACKEND:
1. Receive streaming tokens
2. Forward to frontend via SSE
3. Save chat history
↓
```

FRONTEND:

1. Receive tokens
2. Render markdown
3. Display to user

↓

USER: Sees answer with source citation

Example 2: Document Update Request

USER: "Add tombstone section to Starfish SOP"

↓

FRONTEND: POST /api/chat

↓

BACKEND:

1. Detect: Document update request
2. Load Starfish SOP file
3. Send to AI with update instructions

↓

AI: Generates updated document

↓

BACKEND:

1. Detect "APPROVAL_REQUIRED: YES"
2. Create approval request
3. Save to Approvals/YYYY-MM-DD.md
4. Save temp file to Temp/
5. Broadcast SSE event

↓

FRONTEND:

1. Display updated document
2. Show approval notice
3. Update vault panel (via SSE)

↓

ADMIN: Reviews and approves

↓

BACKEND:

1. Update original SOP file
2. Update temp file status
3. Create final version in Updated Doc/
4. Broadcast SSE event

↓

FRONTEND: Shows success notification

Network Communication

HTTP Endpoints:

Method	Endpoint	Purpose
GET	/	Serve frontend HTML
GET	/api/vault	List all vault files

GET	/api/file?path=...	Get file content
POST	/api/chat	Send message to AI
GET	/api/events	SSE for live updates
GET	/api/network-info	Get server IPs
POST	/api/approve	Approve update request
POST	/api/reject	Reject update request
GET	/api/approvals?date=...	Get approval requests

Request/Response Format:

Chat Request:

```
{
  "message": "Is blk correct?",
  "context": "... file content ...",
  "history": [
    { "role": "user", "content": "previous question" },
    { "role": "assistant", "content": "previous answer" }
  ],
  "selectedFile": "Starfish Modular Title Quality Audits.md",
  "chatId": "2026-05-21/chat_1779335188447.md"
}
```

Chat Response (SSE Stream):

```
data: {"token":"No"}
data: {"token":","}
data: {"token":" publishing"}
data: {"token":" \"blk\""}
...
data: {"done":true,"history":[...],"chatFile":"..."}
```

 Security Considerations

Current Security:

1. Path Traversal Protection:

```
if (!fullPath.startsWith(OBSIDIAN_DB_PATH)) {
  return res.status(403).json({ error: 'Access denied' });
}
```

2. CORS Enabled:

```
app.use(cors()); // Allows cross-origin requests
```

3. Admin Authentication:

```
const ADMIN_USERS = {
  rsuprith: '1234'
};

function validateAdmin(username, password) {
  return ADMIN_USERS[username] && ADMIN_USERS[username] === password;
}
```

Security Limitations:

⚠️ **No encryption** - HTTP only (not HTTPS) ⚠️ **Hardcoded credentials** - Admin password in code ⚠️ **No rate limiting** - Can be abused ⚠️ **No input sanitization** - Potential XSS ⚠️ **Groq API key exposed** - In source code

 Performance Characteristics

Response Times:

Operation	Time	Notes
Load vault files	100-500ms	Depends on file count
Load single file	10-50ms	Depends on file size
Groq API response	2-5 seconds	Fast, cloud-based
Ollama response	30-60 seconds	Slow, local processing
Save chat history	10-50ms	File write operation
SSE broadcast	<10ms	In-memory operation

Resource Usage:

Component	CPU	RAM	Disk	Network
Frontend	Low	50-100MB	None	Low
Backend	Low	100-200MB	Low	Medium
Groq API	None	None	None	High
Ollama	High	4-8GB	4GB	None

 Summary

How It All Works Together:

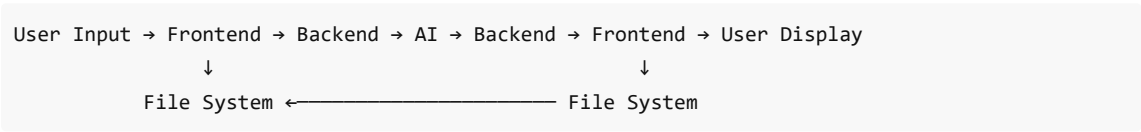
1. **Frontend** (HTML/JS) provides the user interface

- 2. **Backend** (Node.js/Express) handles business logic
- 3. **Groq API** (cloud) provides fast AI responses
- 4. **Ollama** (local) provides fallback AI
- 5. **File System** stores documents and chat history
- 6. **SSE** provides real-time updates

Key Technologies:

- **Frontend:** HTML, CSS, JavaScript, marked.js, highlight.js
- **Backend:** Node.js, Express.js, axios, fs-extra
- **AI:** Groq API (llama-3.1-8b-instant), Ollama (mistral)
- **Communication:** HTTP REST API, Server-Sent Events (SSE)
- **Storage:** File system (Markdown files)

Data Flow:



Your system is a well-architected, full-stack application with intelligent AI integration, real-time updates, and comprehensive document management! 🎉